S.No: 7	Exp. Name: ***Bellman-Ford algorithm***	Date:2026-07-10

Aim:

Bellman-Ford is a shortest path algorithm used to find the minimum distance from a single source vertex to all other vertices in a weighted directed graph. Unlike Dijkstra's algorithm, Bellman-Ford can handle negative edge weights and can also detect negative weight cycles.

Your task is to implement the **Bellman-Ford algorithm** and print the shortest distance and path from the given source vertex to every other vertex.

Input Format:

- The first line contains an integer V, the number of vertices in the graph.
- The second line contains an integer E, the number of edges in the graph.
- Next E lines: Each line contains three integers u , v , w - representing an edge from vertex u to vertex v with weight w.
- Last line contains an integer src, the source vertex from which shortest paths will be calculated.

Output Format:

- For each vertex v from 1 to V (except the source), the program prints:

If a node is reachable:
the program prints:

```
vertex shortest_distance path
```

Example:

```
4 6 1->2->4
```

Meaning:

- Vertex = 4
- Shortest distance from source = 6
- Path = 1 → 2 → 4

If a node is NOT reachable:
the program prints:

```
vertex INF None
```

Example:

```
5 INF None
```

Meaning:

- Vertex = 5
- Distance = INF (infinite)
- No path exists

If a negative cycle exists:
The program prints:

```
Negative cycle detected
```

Constraints:

- $1 <= V <= 1000$

• Use Bellman-Ford algorithm.

Note:

• Vertices are numbered from 1 to V.

• To run your custom test cases, strictly map your input and output layout with the visible test cases.

Source Code:

CTC79166.c

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define INF 1e9

typedef struct {
    int src, dest, weight;
} Edge;

// FIXED: Correctly passes the parent node up the chain and maintains 'src'
void printPath(int parent[], int vertex, int src) {
    if (vertex == src) {
        printf("%d", src);
        return;
    }
    printPath(parent, parent[vertex], src);
    printf("->%d", vertex);
}

void bellmanFord(int V, int E, Edge edges[], int src) {
    int *dist = (int *)malloc((V + 1) * sizeof(int));
    int *parent = (int *)malloc((V + 1) * sizeof(int));

    for (int i = 1; i <= V; i++) {
        dist[i] = INF;
        parent[i] = -1;
    }
    dist[src] = 0;

    // Relax all edges V - 1 times
    for (int i = 1; i <= V - 1; i++) {
        for (int j = 0; j < E; j++) {
            int u = edges[j].src;
            int v = edges[j].dest;
            int weight = edges[j].weight;

            if (dist[u] != INF && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                parent[v] = u;
            }
        }
    }

    // Check for negative-weight cycles
```


ID: 25070521290
2025-29-CSE-E
Symbiosis Institute of Technology | SIT Nagpur

```
    bool negativeCycle = false;
    for (int j = 0; j < E; j++) {
        int u = edges[j].src;
        int v = edges[j].dest;
        int weight = edges[j].weight;

        if (dist[u] != INF && dist[u] + weight < dist[v]) {
            negativeCycle = true;
            break;
        }
    }

    if (negativeCycle) {
        printf("Negative cycle detected\n");
    } else {
        for (int i = 1; i <= V; i++) {
            if (i == src) continue;

            if (dist[i] >= INF / 2) {
                printf("%d INF None\n", i);
            } else {
                printf("%d %d ", i, dist[i]);
                printPath(parent, i, src);
                printf("\n");
            }
        }
    }

    free(dist);
    free(parent);
}

int main() {
    int V, E;
    if (scanf("%d", &V) != 1) return 0;
    if (scanf("%d", &E) != 1) return 0;

    Edge *edges = (Edge *)malloc(E * sizeof(Edge));
    for (int i = 0; i < E; i++) {
        if (scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight) != 3)
{
            free(edges);
            return 0;
        }
    }

    int src;
    if (scanf("%d", &src) != 1) {
        free(edges);
        return 0;
    }

    bellmanFord(V, E, edges, src);

    free(edges);
```

```
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
7
1 2 17
2 3 25
3 4 1
4 2 -24
2 5 15
1 4 40
5 3 9
1
2 16 1->4->2
3 40 1->4->2->5->3
4 40 1->4
5 31 1->4->2->5

Test Case - 2
User Output
3
3
1 2 4
2 3 -10
3 1 3
1
Negative cycle detected

ID: 25070521290